

# Производные типы данных в MRI

# Производные типы данных в MPI.

Производный тип данных в MPI - описание набора значений предусмотренного в MPI типа, значения могут не располагаться непрерывно по памяти:

- Задание типа в MPI принято осуществлять при помощи **карты типа (type map)** в виде последовательности описаний входящих в тип значений, каждое отдельное значение описывается указанием типа и смещения адреса месторасположения от некоторого базового адреса:

**TypeMap** = {(type<sub>0</sub>, disp<sub>0</sub>), (type<sub>1</sub>, disp<sub>1</sub>), ... , (type<sub>n-1</sub>, disp<sub>n-1</sub>)}

- Часть карты типа с указанием только типов значений именуется в MPI **сигнатурой типа**:

**TypeSignature** = {type<sub>0</sub>, type<sub>1</sub>, ... , type<sub>n-1</sub>}

# Производные типы данных в MPI.

Пример:

- Пусть в сообщение должны входить значения переменных:

```
double a; /* адрес 24 */  
double b; /* адрес 40 */  
int     n; /* адрес 48 */
```

- Тогда производный тип для описания таких данных должен иметь карту типа следующего вида:

```
{(MPI_DOUBLE, 0),  
 (MPI_DOUBLE, 16),  
 (MPI_INT, 24)  
}
```

# Производные типы данных в MRI.

Способы конструирования производных типов данных:

- **Непрерывный** способ позволяет определить непрерывный набор элементов существующего типа как новый производный тип,
- **Векторный** способ обеспечивает создание нового производного типа как набора элементов существующего типа, между элементами которого существуют регулярные промежутки по памяти. При этом, размер промежутков задается в числе элементов исходного типа,
- **Индексный** способ отличается от векторного метода тем, что промежутки между элементами исходного типа могут иметь нерегулярный характер,
- **Структурный** способ обеспечивает самое общее описание производного типа через явное указание карты создаваемого типа данных.

# Производные типы данных в MPI.

Непрерывный способ конструирования:

```
int MPI_Type_contiguous (int count, MPI_Datatype oldtype,  
                        MPI_Datatype *newtype);
```

- Как следует из описания, новый тип `newtype` создается как `count` элементов исходного типа `oldtype`. Например, если исходный тип данных имеет карту типа:

```
{(MPI_INT, 0), (MPI_DOUBLE, 8)}
```

то вызов функции `MPI_Type_contiguous` с параметрами:

```
MPI_Type_contiguous (2, oldtype, &newtype);
```

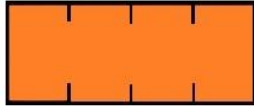
приведет к созданию типа данных с картой типа:

```
{(MPI_INT, 0), (MPI_DOUBLE, 8),  
 (MPI_INT, 16), (MPI_DOUBLE, 24)}
```

# Производные типы данных в MPI.

Непрерывный способ конструирования:

oldtype = MPI\_REAL



count = 4

newtype



# Производные типы данных в MPI.

Векторный способ конструирования:

- При векторном способе новый производный тип создается как набор блоков из элементов исходного типа, при этом между блоками могут иметься регулярные промежутки по памяти.

```
int MPI_Type_vector (int count,  
                    int blocklen,  
                    int stride,  
                    MPI_Datatype oldtype,  
                    MPI_Datatype *newtype),
```

где

- `count` – количество блоков,
- `blocklen` – размер каждого блока,
- `stride` – количество элементов, расположенных между двумя соседними блоками,
- `oldtype` – исходный тип данных,
- `newtype` – новый определяемый тип данных.

# Производные типы данных в MPI.

Векторный способ конструирования:

- Если интервалы между блоками задаются в байтах, а не в элементах исходного типа данных, следует использовать функцию:

```
int MPI_Type_hvector (int count,  
                      int blocklen,  
                      MPI_Aint stride,  
                      MPI_Datatype oldtype,  
                      MPI_Datatype *newtype);
```



# Производные типы данных в MPI.

Векторный способ конструирования:

- Создание производных типов для описания подмассивов многомерных массивов.

```
int MPI_Type_create_subarray (int ndims, int *sizes,  
                             int *subsizes, int *starts, int order,  
                             MPI_Datatype oldtype, MPI_Datatype *newtype),
```

где

- `ndims` – размерность массива,
- `sizes` – количество элементов в каждой размерности исходного массива,
- `subsizes` – количество элементов в каждой размерности определяемого подмассива,
- `starts` – индексы начальных элементов в каждой размерности определяемого подмассива,
- `order` – параметр для указания необходимости переупорядочения,
- `oldtype` – тип данных элементов исходного массива,
- `newtype` – новый тип данных для описания подмассива.

# Производные типы данных в MPI.

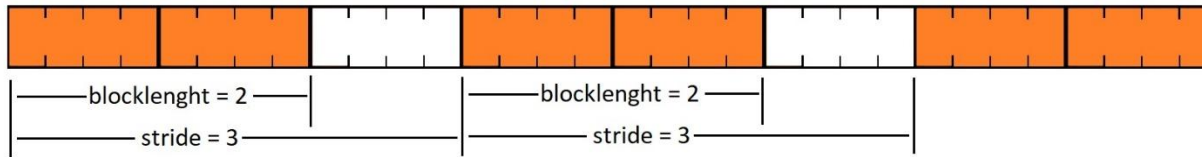
Векторный способ конструирования:

oldtype = MPI\_REAL



count = 3, blocklength = 2, stride = 3

newtype



# Производные типы данных в MPI.

Индексный способ конструирования:

- Новый производный тип создается как набор блоков разного размера из элементов исходного типа, при этом между блоками могут иметься разные промежутки по памяти.

```
int MPI_Type_indexed (int count,  
                     int blocklens[],  
                     int indices[],  
                     MPI_Datatype oldtype,  
                     MPI_Datatype *newtype),
```

где

- `count` — количество блоков,
- `blocklens` — количество элементов в каждом блоке,
- `indices` — смещение каждого блока от начала типа (в количестве элементов исходного типа),
- `oldtype` — исходный тип данных,
- `newtype` — новый определяемый тип данных.

# Производные типы данных в MPI.

Индексный способ конструирования:

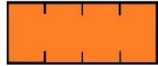
- Если интервалы между блоками задаются в байтах, а не в элементах исходного типа данных, следует использовать функцию:

```
int MPI_Type_hindexed (int count,  
                        int blocklens[],  
                        MPI_Aint indices[],  
                        MPI_Datatype oldtype,  
                        MPI_Datatype *newtype);
```

# Производные типы данных в MPI.

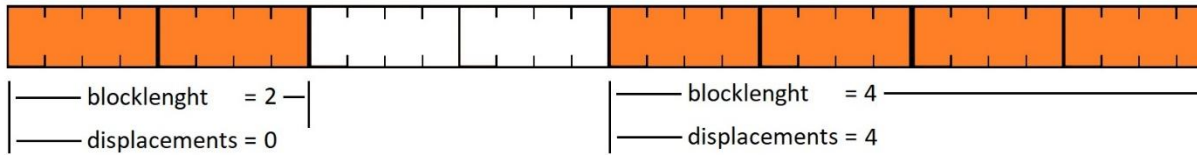
Индексный способ конструирования:

oldtype = MPI\_REAL



count = 2, blocklength = (2, 4), displacements = (0, 4)

newtype



# Производные типы данных в MPI.

Структурный способ конструирования:

- Является самым общим методом конструирования производного типа данных при явном задании соответствующей карты типа:

```
int MPI_Type_struct (int count,  
                    int blocklens[],  
                    MPI_Aint indices[],  
                    MPI_Datatype oldtypes[],  
                    MPI_Datatype *newtype),
```

где

- `count` — количество блоков,
- `blocklens` — количество элементов в каждом блоке,
- `indices` — смещение каждого блока от начала типа (в байтах),
- `oldtypes` — исходные типы данных в каждом блоке в отдельности,
- `newtype` — новый определяемый тип данных.

# Производные типы данных в MPI.

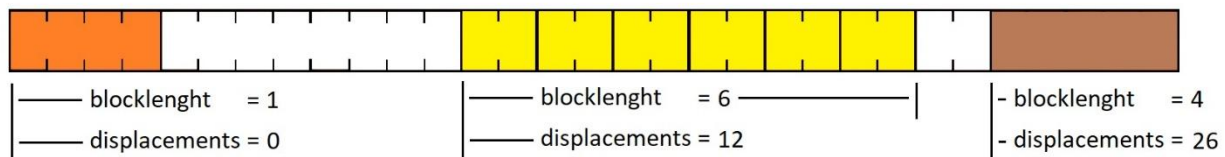
Структурный способ конструирования:

oldtypes = (MPI\_INT, MPI\_SHORT, MPI\_CHAR)



count = 3, blocklength = (1, 6, 4), displacements = (0, 12, 26)

newtype



# Производные типы данных в MPI.

Объявление производных типов и их удаление:

- Перед использованием созданный тип **должен быть объявлен**:

```
int MPI_Type_commit (MPI_Datatype *type);
```

- При завершении использования производный тип должен быть аннулирован при помощи функции:

```
int MPI_Type_free (MPI_Datatype *type);
```



# Где посмотреть?

<http://rsusu1.rnd.runnet.ru/tutor/method/m2/page15.html>

# Производные типы данных в MRI